Week 3: Conditionals and Loops

ID1063 • Introduction to Programming in C

Instructor: Rakesh Venkat

August 2026

Recap from Week 2

- scanf to get input from the user
- Variable types and updating variables (e.g. a=a+1)
- Arithmetic expressions and precedence
- Relational operators compare two values (x>y, x==y, x<y, x<=y, x!=y)
- Logical operators combine or negate conditions.
  - In C, zero is false; a nonzero value is true.

A few announcements

- Attend Lab sessions! These are the best place where you can learn and get help from the TAs or instructors. We are here to help!

- If you are finding the content difficult, **please let us know**: fill the google form that will be circulated separately and mention what you are finding hard to follow. We will plan to cover those parts again if needed.

- Moodle is back up. Please enroll on moodle with the course password:

id1063-2026

This week

- Decisions using conditionals
- Introduction to loops

Conditionals and decision making

The if statement

```
x = 0;              /* assignment: store zero */
if (x == 0)
{
    printf("x is zero\n");
}
```

logical expn

what to do if expn is true

```
if (logical expression)
{
    statements;
}
```

If block

```
int num;
scanf("%d", &num);

if (num < 0)
{
    num = -num;
}

printf("%d\n", num);
```

Q: What does the above program do?

Logical operators




```
int age;
scanf("%d", &age);

if (age >= 13 && age <= 19)
{
    printf("Teenage range\n");
}
```

```
if ( age < 13  || age > 19 )
{   printf ( "not a teen\n" ); ->
}
```

Q: What if we want to print “Not a teen” for other ages?

The If-Else statement

```
int n;
scanf("%d", &n);

if (n % 2 == 0)
{
    printf("E\n");
}
else
{
    printf("O\n");
}
```

% = remainder

Q: What does the above program do?

Only one branch runs

- The condition n % 2 == 0 is either true or false.
- The if and else blocks are **alternatives**, not two separate tests.
- The statement after the conditional runs in both cases.
- Braces make the intended blocks visible.

```
int n;
scanf("%d", &n);

if (n % 2 == 0)
{
    printf("E\n");
}
else
{
    printf("O\n");
}
```

How If-Else works

Evaluate the condition once.

If it is true, run the if block.

Otherwise, run the else block.

After either block, continue with the next statement.

```
if (condition)
{
        statements to run if condition is true
}
else
{
        statements to run otherwise
}
```

Compare two integers

```
int a, b;
scanf("%d %d", &a, &b);

if (a > b)
{
    printf("%d is larger\n", a);
}
else
{
    printf("%d is larger\n", b);
}
```

a = 7 b = 7

Q: What happens when a and b are equal?

Chaining else-if

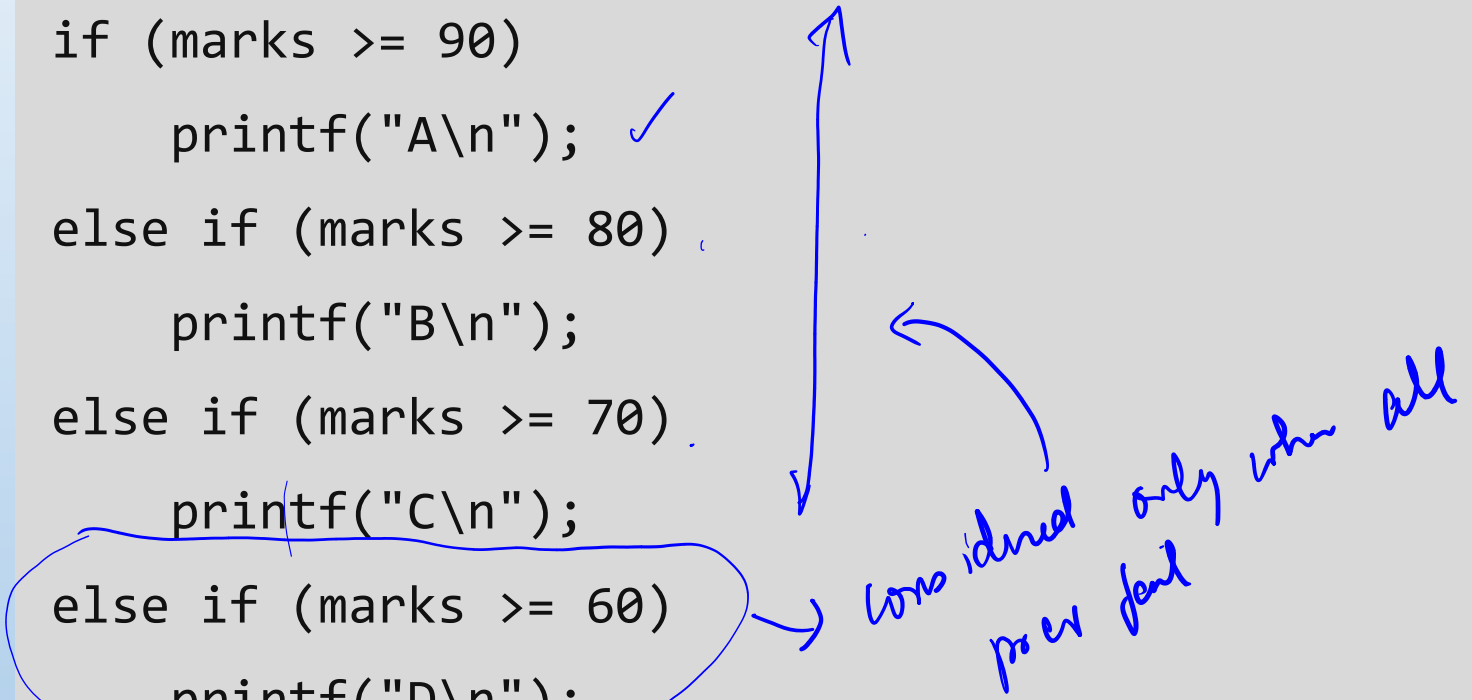
```
int score;
scanf("%d", &score);

if (score >= 90)
    printf("Excellent\n");
else if (score >= 60)
    printf("Pass\n");
else
    printf("Needs improvement\n");
```

IMPORTANT: Note the order of the conditions imposed!

Example: assign a grade

```
if (marks >= 90)
    printf("A\n");
else if (marks >= 80)
    printf("B\n");
else if (marks >= 70)
    printf("C\n");
else if (marks >= 60)
    printf("D\n");
else
    printf("F\n");
```



The else-if pattern

```
if (condition_1)
{
    /* first case */
}
else if (condition_2)
{
    /* second case */
}
else if (condition_3)
{
    /* third case */
}
else
{
    /* every remaining case */
}
```

Conditions are tested from top to bottom.

As soon as one condition is true, its block runs.

Every later branch in the same chain is skipped.

At most one branch in the chain executes.

Place restrictive cases before broader cases

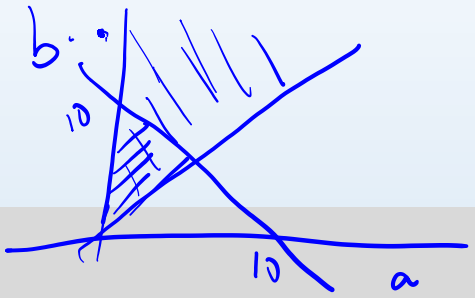
- For descending thresholds, test the highest threshold first.
- After marks >= 90 is false, the next test already knows marks < 90.
- That is why the B branch only needs marks >= 80.
- The chain itself carries information from earlier failed tests.

```
if (marks >= 90)
    printf("A\n");
else if (marks >= 80)
    printf("B\n");
else if (marks >= 70)
    printf("C\n");
else if (marks >= 60)
    printf("D\n");
else
    printf("F\n");
```

Comparing integers

```
if (a > b)
{
    printf("%d is larger\n", a);
}
else if (b > a)
{
    printf("%d is larger\n", b);
}
else
{
    printf("The values are equal\n");
}
```

Find the output




```
if (a > b)
{
    printf("%d is larger\n", a);
}
else if (b+a > 10)
{
    printf(“Sum is greater than 10 \n");
}
else
{
   printf("The values are equal\n");
}
```

a=3, b=3

a=6, b=6

when a>b is false and b+a>10 is false

eg: a=3, b=4

Q: For what values of a,b is the third statement printed?

A: all values where $a \leq b$ and $a + b \leq 10$

→ for $a \leq b$ and $b+a \leq 10$

Data validation within if-else

```
if (marks < 0 || marks > 100)
{
    printf("Invalid marks\n");
}
else if (marks >= 90)
{
    printf("A\n");
}
else if (marks >= 80)
{
    printf("B\n");
}
/* Continue with the remaining grades. */
```

Triangle classification

- Given 3 lengths a,b,c, classify the type of the triangle with side lengths a,b,c

a, b, c form a Δ iff:

$a+b \geq c$ $\quad a>0, b>0, c>0$

and $b+c>a$

and $c+a>b$

```
if (a<=0 || b<=0 || c<=0)
{ printf ("invalid non positive input")
}
else if((a+b<=c) || (b+c)<=a || (c+a)<=b)
 { printf ("Triangle ineq violated");
 }
```

Triangle classification

```
int invalid = (a + b <= c) ||
              (a + c <= b) ||
              (b + c <= a);

if (invalid)
{
    printf("Not a triangle\n");

}
else if ((a == b) && (b == c))
{
    printf("Equilateral\n");
}
else if (a == b || a == c || b == c)
{
   printf("Isosceles\n");

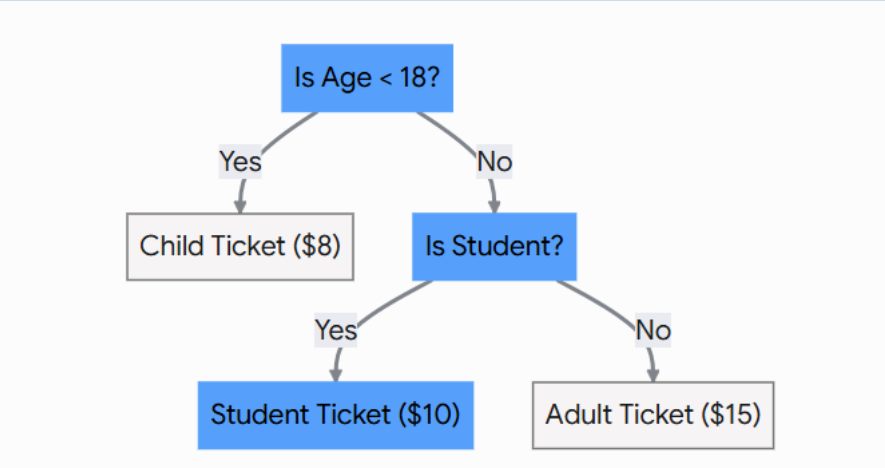
}
else

{
    printf("Scalene\n");
}
```

if (abc, invalid = 0
else invalid = 1

Nested conditionals

```
if (condition1) {
    if (condition2) {
        // executes if both condition1 and condition2 are true
    } else {
        // executes if condition1 is true, condition2 is false
    }
} else {
    // executes if condition1 is false
}
```



A conditional can contain another conditional

The outer condition chooses a broad case.

The inner condition refines only that case.

Use nesting when the second question depends on the first answer.

Indentation should show the structure clearly.

Example: ticket price

```
if (age < 12)
{
    price = 8;
}
else
{
    price = 15;

    if (is_student)
    {
        price = 10;
    }
}

printf("Price = %d\n", price);
```

An alternate way to check triangles

```
if (a <= 0 || b <= 0 || c <= 0 ||
    a + b <= c || a + c <= b || b + c <= a)
{
    printf("Not a triangle\n");
}
else
{
    if (a == b && b == c)
        printf("Equilateral\n");
    else if (a == b || b == c || a == c)
        printf("Isosceles\n");
    else
        printf("Scalene\n");
}
```

```
int invalid = (a + b <= c) ||
              (a + c <= b) ||
              (b + c <= a) || a<=0 ||b<=0 ||c<=0;

if (invalid)
{
    printf("Not a triangle\n");

}
else if (a == b && b == c)
{
    printf("Equilateral\n");
}
else if (a == b || a == c || b == c)
{
   printf("Isosceles\n");

}
else

{
    printf("Scalene\n");
}
```

Conditionals to guard

```
int numerator, denominator;
scanf("%d %d", &numerator, &denominator);

if (denominator != 0)
{
    printf("Quotient = %d\n",
           numerator / denominator);
}
else
{
    printf("Cannot divide by zero\n");
}
```

Logical operators use short-circuit evaluation

For A && B: if A is false, B is not evaluated.

For A || B: if A is true, B is not evaluated.

Evaluation proceeds from left to right.

This can avoid an invalid operation - if the protective test comes first.

A protective test must come first

```
if (denominator != 0 &&
    numerator / denominator > 2)
{
    printf("Large quotient\n");
}

/* If denominator is zero, the division is skipped. */
```

Condition order matters

```
/* Unsafe order */
if (numerator / denominator > 2 &&
    denominator != 0)
{
    printf("Large quotient\n");
}

/* The division may happen before zero is checked. */
```

Question: Reorder the two parts to make the condition safe.

Common conditional errors

Checking ranges

```
/* Correct: marks is between 0 and 100. */
if (0 <= marks && marks <= 100)
{
    printf("Valid\n");
}

/* Incorrect mathematical shorthand in C. */
if (0 <= marks <= 100)
{
    printf("This test does not mean what it appears to mean.\n");
}
```

Reason given in next slide

Error in range checking explained

```
int marks = 150;


if (0 <= marks <= 100)
    printf("Valid\n");


/* First: 0 <= marks gives 0 or 1.
   Then that result is compared with 100. So this program will
print Valid. */
```

Assignment is not equality

```
int x = 5;


if (x = 0)
    printf("Zero\n");
else
    printf("Nonzero\n");


/* x = 0 stores zero in x.
   x == 0 compares x with zero. */


Question: What is x after the condition is evaluated?
```

A stray semicolon ends the if statement

```
if (temperature > 40);
{
    printf("Heat warning\n");
}
```

Q: what is the output?

An else matches the nearest unmatched if

```
if (logged_in)
    if (is_admin)
        printf("Admin page\n");
    else
        printf("User page\n");
```

Question: Which if owns the else? Add braces to show the intended structure.

Debug this conditional

```
int age;
scanf("%d", &age);

if age >= 18
{
    printf("Adult\n")
}
else if (0 <= age < 18)
{
    printf("Minor\n");
}
else
{
    printf("Invalid age\n");
}
```

Question: Find the syntax error and the logic error.

One corrected version

```
int age;
scanf("%d", &age);

if (age < 0)
{
    printf("Invalid age\n");
}
else if (age >= 18)
{
    printf("Adult\n");
}
else
{
    printf("Minor\n");
}
```